

Universidad Técnica Estatal de Quevedo

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

Asignatura:
Aplicaciones Web

TA – Proyecto Fin de Curso: Entrega 1A
Planificación y Diseño

Proyecto: BIOPET

Integrantes:
Beltrán Montiel Fred Adrián - 1250300876
Mariscal Cabrera Jaime Josué - 1250710835
Taípe Mora Zaida Melissa - 0604907956

Docente:
Ing. Guerrero Ulloa Gleiston Cicerón

Periodo Académico:
2025-2026 SPA

Fecha de Entrega:
4 de junio de 2026

URL del Repositorio: ([Git](#))

Índice

1. Descripción del sistema web propuesto	2
2. Revisión y actualización de requisitos del semestre anterior	2
2.1. Resumen del trabajo previo	2
2.2. Tabla de adaptacion de requisitos	3
2.3. Nuevos requisitos específicos del contexto web	4
3. Lista consolidada de Requisitos Funcionales (RF)	4
4. Lista consolidada de Requisitos No Funcionales (RNF)	6
5. Arquitectura de la aplicación web	6
5.1. Diagrama de arquitectura (C4)	7
5.2. Descripción de capas y tecnologías	8
5.3. ADR-001: Selección del lenguaje del servidor [6]	9
6. Modelo de base de datos	9
6.1. Modelo Entidad-Relación (MER)	10
6.2. Diccionario de datos	10
6.3. Script DDL SQL (Fragmento Core)	12
7. Diseño de interfaz: Wireframes	18
8. Plan de trabajo y roles	21
8.1. Cronograma de Hitos	21
8.2. Asignación de roles	23
9. Estructura del repositorio	23

1. Descripción del sistema web propuesto

Problema: En el contexto de las clínicas veterinarias de pequeña y mediana escala en Ecuador, la gestión de información clínica y administrativa se realiza predominantemente mediante procesos manuales o herramientas digitales aisladas, como hojas de cálculo y aplicaciones de mensajería. Esta situación genera fragmentación de datos, duplicidad de registros e inconsistencias que dificultan el seguimiento continuo de la salud animal. Durante el proceso de elicitación de requisitos del semestre anterior, entrevistas realizadas a tres veterinarios y dos auxiliares confirmaron que la información clínica frecuentemente se encuentra incompleta o dispersa, lo que retrasa la atención y complica la toma de decisiones. Complementariamente, encuestas aplicadas a quince dueños de mascotas evidenciaron que la comunicación con las clínicas es limitada y poco eficiente. Con base en ese análisis, se proyectó que la implementación de una solución integral permitiría una reducción aproximada del 40 % en el tiempo dedicado a la gestión administrativa.

Público Objetivo: BIOPET es un sistema web integral orientado a centralizar la información clínica y administrativa de clínicas veterinarias de pequeña y mediana escala. El sistema contempla tres roles principales: el Veterinario, quien registra diagnósticos, gestiona historiales clínicos y recibe sugerencias del sistema; el Administrador, quien supervisa roles, inventario, facturación y la operatividad general; y el Dueño de Mascota, quien accede al seguimiento preventivo de su animal, recibe alertas y agenda citas.

Justificación web y alcance: La adopción de una arquitectura web elimina la dependencia de instalaciones locales, permitiendo el acceso en tiempo real a la información clínica desde cualquier dispositivo con conexión a internet. Adicionalmente, una arquitectura basada en la nube facilita la recepción de datos provenientes de dispositivos IoT y el consumo de servicios de Inteligencia Artificial para apoyo clínico [2], sin requerir infraestructura física adicional en la clínica.

Para el presente semestre se implementará el núcleo funcional del sistema, compuesto por los módulos de gestión clínica, historial médico, citas veterinarias, facturación digital, y los endpoints necesarios para la recepción de telemetría IoT. Estos módulos fueron seleccionados porque corresponden a las funcionalidades clasificadas como *Must Have* en la priorización MoSCoW realizada el semestre anterior, es decir, aquellas sin las cuales el sistema no puede operar de forma básica. La integración de Inteligencia Artificial queda postergada para una fase posterior, dado que su complejidad técnica podría retrasar la entrega del núcleo operativo.

2. Revisión y actualización de requisitos del semestre anterior

2.1. Resumen del trabajo previo

Durante el semestre anterior se desarrolló la especificación de requisitos del proyecto "Sistema Integral de Gestión Veterinaria y Cuidado Inteligente de Mascotas", orientado a mejorar la gestión clínica y administrativa de clínicas veterinarias mediante la centralización de información, el seguimiento inteligente de mascotas y el apoyo a la toma de decisiones.

La elicitación de requisitos se realizó mediante entrevistas semiestructuradas a veterinarios y auxiliares veterinarios, así como encuestas dirigidas a dueños de mascotas. Posteriormente, los requisitos fueron analizados, clasificados y documentados siguiendo los lineamientos de la norma ISO/IEC/IEEE 29148 y el enfoque GORE.

Como resultado, se documentaron 15 requisitos funcionales (RF-01 al RF-15) y 4 requisitos no funcionales (RNF-01 al RNF-04). El producto generado fue un documento de Especificación de Requisitos de Software (SRS), complementado con modelos UML, escenarios, criterios de validación y matriz de trazabilidad.

Para la elaboración del proyecto se utilizaron herramientas como Overleaf para la documentación en LaTeX, Visual Paradigm para el modelado UML, Google Meet para entrevistas y encuestas, y Google Drive como repositorio colaborativo.

2.2. Tabla de adaptacion de requisitos

Tabla 1: Adaptación de requisitos funcionales del semestre anterior

Código	Requisito original	Estado	Adaptación para la aplicación web
RF-01	Registro de dueño de mascota	Adapta	Formulario web con validación cliente y servidor.
RF-02	Actualización y consulta de dueño de mascota	Adapta	Gestión mediante vistas CRUD y API REST.
RF-03	Administración del historial clínico	Adapta	Dashboard web para consulta y administración del historial.
RF-04	Registro de atención médica	Adapta	Formularios web conectados a servicios backend REST.
RF-05	Prescripción de medicamentos	Adapta	Registro digital asociado a la atención médica desde interfaz web.
RF-06	Gestión de citas veterinarias	Mantiene	Implementación mediante API REST y calendario web interactivo.
RF-07	Notificaciones de seguimiento	Adapta	Envío de correos electrónicos y notificaciones desde el backend.
RF-08	Seguimiento mediante dispositivo de localización	Adapta	Integración mediante API ASP.NET Core para recepción de datos IoT.
RF-09	Visualización de ubicación de la mascota	Adapta	Visualización de ubicación en interfaz web mediante mapas.
RF-10	Generación de recomendaciones asistidas por IA	Mantiene	Consumo de servicios de IA desde el backend para generar sugerencias clínicas.
RF-11	Facturación de servicios veterinarios	Adapta	Generación dinámica de comprobantes PDF desde el servidor.
RF-12	Registro de pagos	Adapta	Registro de métodos de pago mediante formularios web, sin pasarelas externas.
RF-13	Control de acceso por roles	Adapta	Implementación de RBAC con autenticación JWT.

Código	Requisito original	Estado	Adaptación para la aplicación web
RF-14	Generación de reportes	Adapta	Exportación dinámica a PDF y Excel desde la aplicación web.
RF-15	Auditoría de operaciones	Mantiene	Registro de eventos y acciones de usuarios en base de datos.

2.3. Nuevos requisitos específicos del contexto web

Tabla 2: Nuevos requisitos funcionales y no funcionales para la aplicación web

Código	Requisito
RNF-WEB-01	La interfaz debe ser responsiva y funcionar correctamente en dispositivos móviles, tablets y equipos de escritorio (320px–1440px).
RNF-WEB-02	Toda comunicación entre cliente y servidor debe realizarse mediante HTTPS.
RNF-WEB-03	La autenticación debe gestionarse mediante tokens JWT con expiración configurable.
RNF-WEB-04	El tiempo de respuesta de las consultas al historial clínico no debe superar los 2 segundos bajo carga normal.
RNF-WEB-05	La aplicación debe ser compatible con Chrome, Firefox y Edge en sus versiones modernas.
RF-WEB-01	El sistema debe permitir autenticación mediante usuario y contraseña.
RF-WEB-02	El sistema debe implementar roles diferenciados (Administrador, Veterinario y Personal Administrativo).
RF-WEB-03	El sistema debe exponer funcionalidades mediante una API REST para permitir la comunicación entre frontend y backend.
RF-WEB-04	El sistema debe permitir el cierre de sesión y la invalidación segura de tokens de acceso.

3. Lista consolidada de Requisitos Funcionales (RF)

Tabla 3: Lista consolidada de requisitos funcionales

ID	Descripción	Prioridad	Criterio de aceptación
RF-01	El sistema web deberá permitir registrar la información de los dueños de mascotas y asociarla con sus respectivas mascotas.	Alta	El registro queda almacenado en la base de datos y la mascota aparece vinculada al propietario registrado.
RF-02	El sistema web deberá permitir actualizar y consultar la información de los dueños de mascotas mediante operaciones CRUD.	Alta	Los cambios realizados sobre un propietario son visibles inmediatamente después de guardar la información.
RF-03	El sistema web deberá permitir almacenar y consultar el historial clínico de cada mascota de forma cronológica.	Alta	Al consultar una mascota, el historial clínico se muestra ordenado por fecha de atención.

ID	Descripción	Prioridad	Criterio de aceptación
RF-04	El sistema web deberá permitir registrar atenciones médicas incluyendo diagnóstico, tratamiento y observaciones.	Alta	Una atención médica registrada queda asociada al historial clínico correspondiente.
RF-05	El sistema web deberá permitir registrar medicamentos prescritos durante una atención médica.	Media	Los medicamentos ingresados quedan vinculados a la atención médica registrada.
RF-06	El sistema web deberá permitir registrar, modificar y consultar citas veterinarias mediante un calendario interactivo.	Alta	Las citas creadas aparecen en el calendario y pueden ser modificadas posteriormente.
RF-08	El backend deberá proporcionar una API para registrar datos enviados por dispositivos de rastreo asociados a las mascotas.	Media	La API recibe datos válidos y los almacena correctamente en la base de datos.
RF-09	El sistema web deberá permitir visualizar la ubicación de una mascota mediante mapas interactivos.	Media	La ubicación registrada se muestra en un mapa junto con la fecha y hora de actualización.
RF-10	El sistema deberá generar recomendaciones clínicas informativas basadas en los datos del historial médico de la mascota.	Media	El veterinario puede visualizar recomendaciones generadas automáticamente para una mascota con historial registrado.
RF-11	El sistema web deberá generar comprobantes de pago digitales en formato PDF.	Media	El usuario puede descargar un archivo PDF válido después de registrar un servicio veterinario.
RF-12	El sistema deberá permitir registrar métodos de pago asociados a un servicio veterinario.	Media	La forma de pago seleccionada queda almacenada junto con el servicio registrado.
RF-13	El sistema deberá controlar el acceso a funcionalidades e información según el rol del usuario autenticado.	Alta	Un usuario solo puede acceder a las opciones permitidas para su rol.
RF-14	El sistema deberá generar reportes estadísticos exportables en formato PDF y Excel.	Media	El usuario puede descargar correctamente los reportes generados en ambos formatos.
RF-15	El sistema deberá registrar las acciones realizadas por los usuarios dentro del sistema.	Media	Cada operación relevante queda registrada con usuario, fecha y hora.
RF-16	El sistema deberá permitir la autenticación mediante usuario y contraseña.	Alta	Un usuario registrado puede iniciar sesión utilizando credenciales válidas.
RF-17	El sistema deberá permitir el cierre de sesión y la invalidación segura de tokens de acceso.	Alta	Después de cerrar sesión, el token deja de permitir el acceso a recursos protegidos.

4. Lista consolidada de Requisitos No Funcionales (RNF)

Tabla 4: Lista consolidada de requisitos no funcionales

ID	Categoría	Descripción	Métrica de verificación
RNF-01	Rendimiento	El tiempo de respuesta de las consultas al historial clínico no deberá superar los 2 segundos bajo carga normal.	95 % de las solicitudes respondidas en ≤ 2 segundos durante pruebas de carga.
RNF-02	Seguridad	Toda comunicación entre cliente y servidor deberá realizarse mediante HTTPS.	Verificación mediante certificado TLS válido y redirección automática de HTTP a HTTPS.
RNF-03	Seguridad	La autenticación deberá gestionarse mediante tokens JWT con expiración configurable.	Inspección de tokens emitidos y validación de expiración automática.
RNF-04	Usabilidad	La interfaz deberá adaptarse correctamente a dispositivos móviles, tablets y escritorio.	Verificación en resoluciones de 320px, 768px y 1440px mediante Chrome DevTools.
RNF-05	Compatibilidad	La aplicación deberá funcionar correctamente en Chrome, Firefox y Edge en versiones modernas.	Ejecución satisfactoria de los casos de prueba en los tres navegadores.
RNF-06	Disponibilidad	El sistema deberá estar disponible durante las demostraciones académicas del semestre.	Verificación operativa durante las semanas de evaluación establecidas por la asignatura.
RNF-07	Seguridad	Las contraseñas de los usuarios deberán almacenarse cifradas mediante algoritmos hash seguros.	Verificación de que no existan contraseñas en texto plano en la base de datos.
RNF-08	Mantenibilidad	La arquitectura deberá estar organizada en capas separadas para presentación, lógica de negocio y acceso a datos.	Revisión del código fuente y estructura del proyecto.

5. Arquitectura de la aplicación web

La arquitectura del sistema evoluciona hacia un modelo multinivel (Multi-Tier) orientado a servicios web modernos [9].

5.1. Diagrama de arquitectura (C4)



Figura 1: Diagrama de Contexto (Nivel 1)

Descripción: Basado en el estándar de diseño arquitectónico C4 [7], el diagrama presenta a BIOPET interactuando con Veterinarios, Administradores y Dueños, y comunicándose con sistemas externos de IoT, IA y correos.

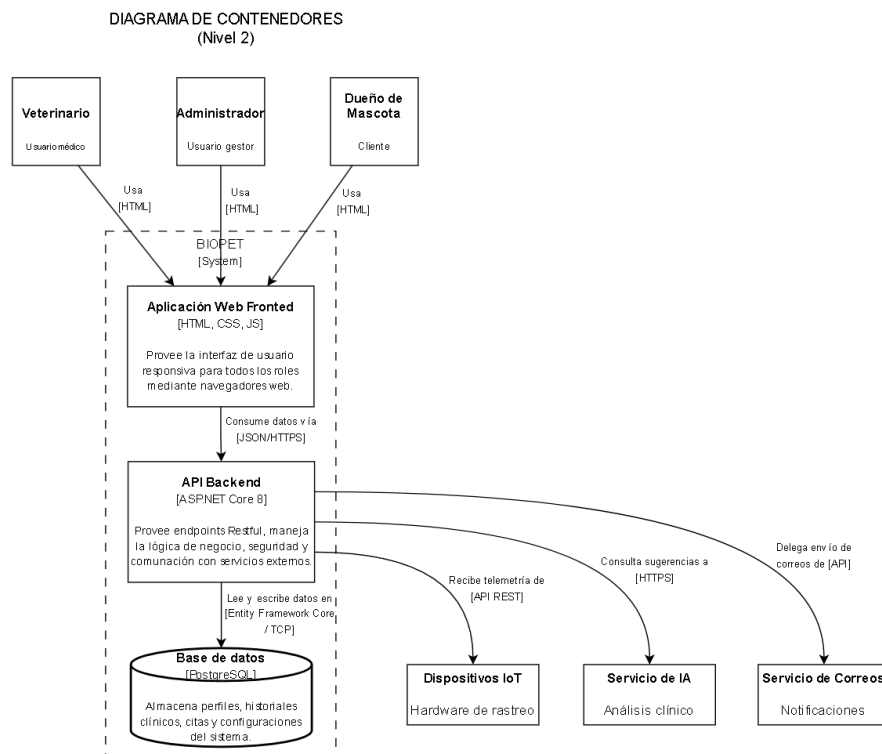


Figura 2: Diagrama de Contenedores (Nivel 2)

Descripción: Se detallan los contenedores internos: la Single Page Application (Frontend), la API REST en ASP.NET Core 8 (Backend) y la base de datos PostgreSQL.

5.2. Descripción de capas y tecnologías

Capa	Tecnología elegida	Justificación
Frontend	HTML5 + CSS3 + JS ES6+	Tecnologías estándar soportadas por todos los navegadores, garantizando compatibilidad inmediata con todos los roles.
Framework CSS	Bootstrap 5	Proporciona componentes responsivos prediseñados que cumplen RNF-01 (adaptación desde 320px).
Backend	ASP.NET Core 8	Seleccionado en ADR-001 por rendimiento superior en APIs REST, y soporte para alta concurrencia IoT.
Framework Data	Entity Framework Core	Permite mapeo objeto-relacional tipado con PostgreSQL, reduciendo errores en desarrollo.
Base de datos	PostgreSQL	Soporte nativo para tipos personalizados (ENUMs definidos en el DDL) e integridad referencial.
Servidor web	Nginx	Actúa como reverse proxy, optimiza rendimiento bajo carga y facilita configuración de HTTPS (RNF-02).
Control de versión	Git + GitHub	Repositorio compartido del equipo con historial verificable.
Despliegue	XAMPP local / Docker	Docker para contenerizar servicios y asegurar consistencia entre entornos de desarrollo y producción.

Tabla 5: Decisión de tecnologías y arquitecturas

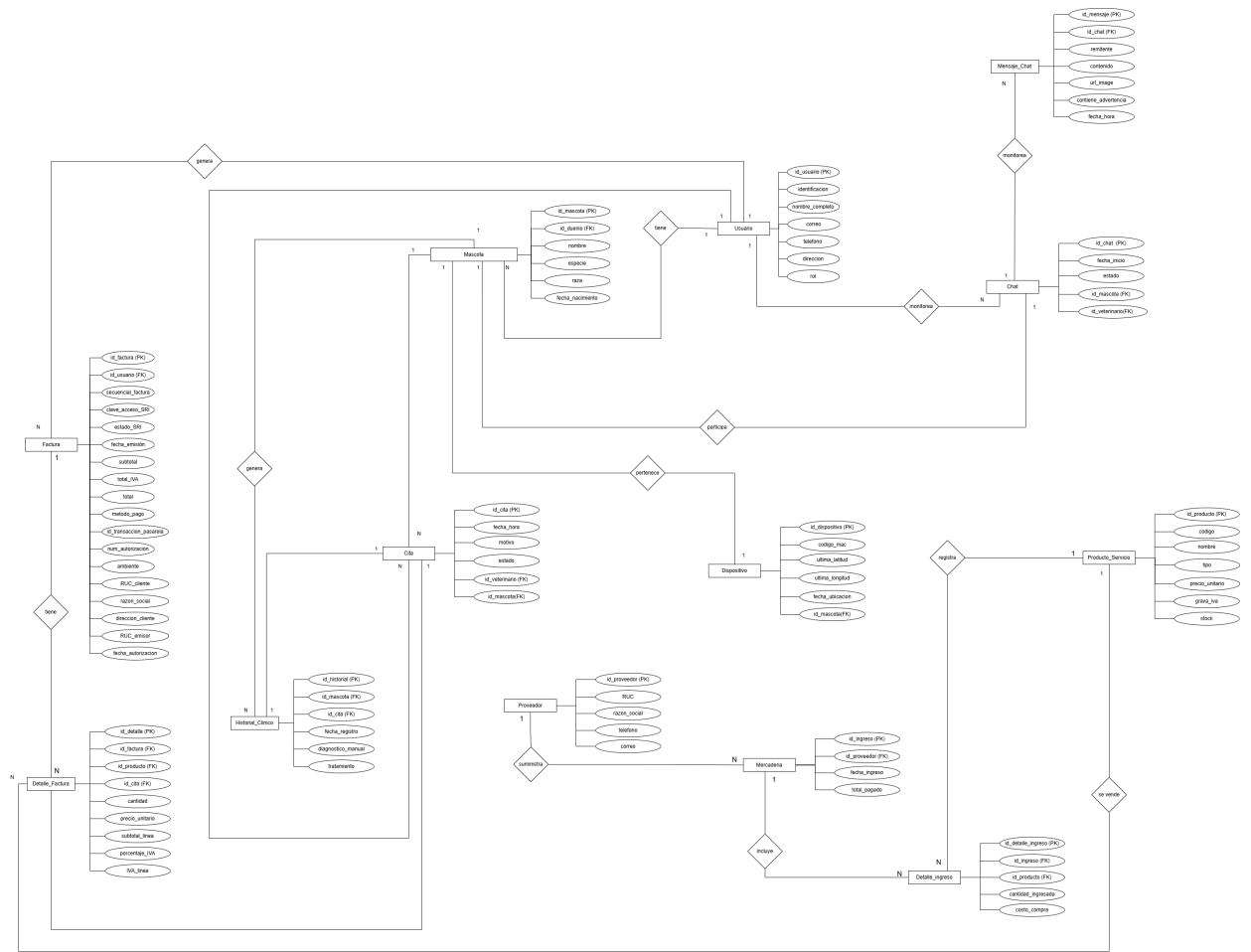
5.3. ADR-001: Selección del lenguaje del servidor [6]

Campo	Contenido
Título	ADR-001: Selección del lenguaje de programación del servidor
Estado	Aceptado
Contexto	El equipo necesita elegir el lenguaje del lado del servidor para implementar BIOPET. Restricciones: tiempo limitado a un semestre, conocimiento en C#/.NET, integración con PostgreSQL, alta concurrencia IoT y soporte para contenedores.
Opciones consideradas	Opción A: PHP 8.2 con Laravel 11. Opción B: ASP.NET Core 8. Opción C: PHP 8.2 sin framework.
Decisión	Se selecciona ASP.NET Core 8 . Ofrece rendimiento superior para APIs REST, manejo asíncrono nativo (async/await) y tipado estático. A diferencia de Laravel, el equipo posee conocimiento previo en C#, lo que reduce la curva de aprendizaje. PHP sin framework fue descartado por carecer de estructura mantenible.
Consecuencias positivas	Alto rendimiento en operaciones concurrentes [1]; tipado estático que facilita detección temprana de errores; integración nativa con EF Core y soporte para JWT.
Consecuencias negativas	Requiere entorno de despliegue compatible (.NET 8 runtime), lo que puede limitar hosting gratuito. La curva de aprendizaje se mitigará con revisión de código y documentación.

6. Modelo de base de datos

El diseño de la base de datos asegura la persistencia, la integridad referencial y la multiplicidad correcta de los registros médicos [3].

6.1. Modelo Entidad-Relación (MER)



[link para mayor visualización \(Aquí\)](#)

Figura 3: Diagrama MER (Usuarios, Mascotas, Citas, Historial Clínico, Facturación)

6.2. Diccionario de datos

Se describe la estructura y restricciones de las entidades principales del núcleo clínico del sistema.

Atributo	Tipo de Dato	Nulo	Restricción	Descripción
id_usuario	INTEGER	No	PK	Identificador único del usuario.
cedula	VARCHAR(13)	No		Número de identificación personal.
nombre_completo	VARCHAR(100)	No		Nombres y apellidos completos.
email	VARCHAR(100)	No	UNIQUE	Correo electrónico de contacto.
telefono	VARCHAR(15)	No		Número telefónico.
direccion	VARCHAR(200)	No		Dirección de residencia.
rol	tipo_rol (ENUM)	No		Rol: Administrador, Veterinario, Auxiliar, Dueño.

Tabla 6: Diccionario de Datos - Entidad: Usuario

Atributo	Tipo de Dato	Nulo	Restricción	Descripción
id_mascota	INTEGER	No	PK	Identificador único de la mascota.
id_duenio	INTEGER	No	FK	Referencia al usuario (dueño de la mascota).
nombre	VARCHAR(50)	No		Nombre de la mascota.
especie	VARCHAR(30)	No		Especie (ej. Canino, Felino).
raza	VARCHAR(50)	No		Raza específica de la mascota.
fecha_nacimiento	DATE	No		Fecha de nacimiento registrada.

Tabla 7: Diccionario de Datos - Entidad: Mascota

Atributo	Tipo de Dato	Nulo	Restricción	Descripción
id_cita	INTEGER	No	PK	Identificador único de la cita médica.
id_mascota	INTEGER	No	FK	Referencia a la mascota paciente.
id_veterinario	INTEGER	No	FK	Referencia al usuario asignado para la atención.
fecha_hora	TIMESTAMP	No		Fecha y hora programada para la consulta.
motivo	VARCHAR(200)	No		Razón principal de la cita médica.
estado	estado_cita (ENUM)	No		Estado: Programada, Completada, Cancelada.

Tabla 8: Diccionario de Datos - Entidad: Cita

Atributo	Tipo de Dato	Nulo	Restricción	Descripción
id_historial	INTEGER	No	PK	Identificador único del registro clínico.
id_mascota	INTEGER	No	FK	Referencia a la mascota atendida.
id_cita	INTEGER	No	FK	Referencia a la cita de origen de la atención.
fecha_registro	TIMESTAMP	No		Fecha y hora exacta de la atención.
diagnostico_manual	TEXT	No		Diagnóstico ingresado por el profesional médico.
tratamiento	TEXT	No		Detalles del tratamiento o prescripción.

Tabla 9: Diccionario de Datos - Entidad: Historial_Clinico

6.3. Script DDL SQL (Fragmento Core)

```
CREATE TYPE "tipo_rol" AS ENUM (
'Administrador',
```

```

'Veterinario',
'Auxiliar',
'Dueno',
'Personal Administrativo'
);
CREATE TYPE "estado_cita" AS ENUM (
'Programada',
'Completada',
'Cancelada'
);
CREATE TYPE "estado_chat" AS ENUM (
'Activo',
'Cerrado'
);
CREATE TYPE "remitente_chat" AS ENUM (
'Dueno',
'IA'
);
CREATE TYPE "tipo_producto" AS ENUM (
'Bien_Fisico',
'Servicio'
);
CREATE TYPE "estado_sri" AS ENUM (
'Creada',
'Recibida',
'Autorizada',
'Rechazada'
);
CREATE TYPE "metodo_pago" AS ENUM (
'Tarjeta',
'Transferencia',
'Efectivo'
);
CREATE TYPE "ambiente" AS ENUM (
'Pruebas',
'Produccion'
);
CREATE TABLE IF NOT EXISTS "Usuario" (
"id_usuario" INTEGER NOT NULL,
"cedula" VARCHAR(13) NOT NULL,
"nombre_completo" VARCHAR(100) NOT NULL,
"email" VARCHAR(100) NOT NULL,
"telefono" VARCHAR(15) NOT NULL,
"direccion" VARCHAR(200) NOT NULL, -- --
"rol" tipo_rol NOT NULL,
PRIMARY KEY("id_usuario")
);
CREATE TABLE IF NOT EXISTS "Mascota" (

```

```

"id_mascota" INTEGER NOT NULL,
"id_duenio" INTEGER NOT NULL,
"nombre" VARCHAR(50) NOT NULL,
"especie" VARCHAR(30) NOT NULL,
"raza" VARCHAR(50) NOT NULL,
"fecha_nacimiento" DATE NOT NULL,
PRIMARY KEY("id_mascota")
);

```

```

CREATE TABLE IF NOT EXISTS "Cita" (
  "id_cita" INTEGER NOT NULL,
  "id_mascota" INTEGER NOT NULL,
  "id_veterinario" INTEGER NOT NULL,
  "fecha_hora" TIMESTAMP NOT NULL,
  "motivo" VARCHAR(200) NOT NULL,
  "estado" estado_cita NOT NULL,
  PRIMARY KEY("id_cita")
);

```

```

CREATE TABLE IF NOT EXISTS "Historial_Clinico" (
  "id_historial" INTEGER NOT NULL,
  "id_mascota" INTEGER NOT NULL,
  "id_cita" INTEGER NOT NULL,
  "fecha_registro" TIMESTAMP NOT NULL,
  "diagnostico_manual" TEXT NOT NULL,
  "tratamiento" TEXT NOT NULL,
  PRIMARY KEY("id_historial")
);

```

```

CREATE TABLE IF NOT EXISTS "Dispositivo_IoT" (
  "id_dispositivo" INTEGER NOT NULL,
  "id_mascota" INTEGER NOT NULL UNIQUE,
  "codigo_mac" VARCHAR(50) NOT NULL,
  "ultima_latitud" NUMERIC(10,8) NOT NULL,
  "ultima_longitud" NUMERIC(10,8) NOT NULL,
  "fecha_ultima_ubicacion" TIMESTAMP NOT NULL,
  PRIMARY KEY("id_dispositivo")
);

```

```

CREATE TABLE IF NOT EXISTS "Chat_Triage" (
  "id_chat" INTEGER NOT NULL,
  "id_mascota" INTEGER NOT NULL UNIQUE,
  "fecha_inicio" TIMESTAMP NOT NULL,
  "estado" estado_chat NOT NULL,
  "id_veterinario" INTEGER NOT NULL,
  PRIMARY KEY("id_chat")
);

```

```

CREATE TABLE IF NOT EXISTS "Mensaje_Chat" (
  "id_mensaje" INTEGER NOT NULL,
  "id_chat" INTEGER NOT NULL,
  "remitente" remitente_chat NOT NULL,
  "contenido" TEXT NOT NULL,
  "url_imagen" VARCHAR(255) NOT NULL,
  "contiene_advertencia" BOOLEAN NOT NULL,
  "fecha_hora" TIMESTAMP NOT NULL,
  PRIMARY KEY("id_mensaje")
);

```

```

CREATE TABLE IF NOT EXISTS "Producto_Servicio" (
  "id_producto" INTEGER NOT NULL,
  "codigo_principal" VARCHAR(25) NOT NULL,
  "nombre" VARCHAR(100) NOT NULL,
  "tipo" tipo_producto NOT NULL,
  "precio_unitario" NUMERIC(10,2) NOT NULL,
  "grava_iva" BOOLEAN NOT NULL,
  "stock" INTEGER NOT NULL,
  PRIMARY KEY("id_producto")
);

```

```

CREATE TABLE IF NOT EXISTS "Factura" (
  "id_factura" INTEGER NOT NULL,
  "id_cliente" INTEGER NOT NULL,
  "secuencial_factura" VARCHAR(17) NOT NULL,
  "clave_acceso_sri" VARCHAR(49) NOT NULL,
  "estado_sri" estado_sri NOT NULL,
  "fecha_emision" TIMESTAMP NOT NULL,
  "subtotal" NUMERIC(10,2) NOT NULL,

```



```

"total_iva" NUMERIC(10,2) NOT NULL,
"total" NUMERIC(10,2) NOT NULL,
"metodo_pago" metodo_pago NOT NULL,
"id_transaccion_pasarela" VARCHAR(100) NOT NULL,
"numero_autorizacion" VARCHAR(49) NOT NULL,
"fecha_autorizacion" TIMESTAMP NOT NULL,
"ambiente" ambiente NOT NULL,
"ruc_cliente" VARCHAR(13) NOT NULL,
"razon_social" VARCHAR(100) NOT NULL,
"direccion_cliente" VARCHAR(200) NOT NULL,
"ruc_emisor" VARCHAR(100) NOT NULL,
PRIMARY KEY("id_factura")
);

```

```

CREATE TABLE IF NOT EXISTS "Detalle_Factura" (
  "id_detalle" INTEGER NOT NULL,
  "id_factura" INTEGER NOT NULL,
  "id_producto" INTEGER NOT NULL,
  "id_cita" INTEGER NOT NULL,
  "cantidad" INTEGER NOT NULL,
  "precio_unitario" NUMERIC(10,2) NOT NULL,
  "subtotal_linea" NUMERIC(10,2) NOT NULL,
  PRIMARY KEY("id_detalle")
);

```

```

CREATE TABLE IF NOT EXISTS "Proveedor" (
  "id_proveedor" INTEGER NOT NULL,
  "ruc" VARCHAR(13) NOT NULL,
  "razon_social" VARCHAR(100) NOT NULL,
  "telefono" VARCHAR(15) NOT NULL,
  "correo" VARCHAR(100) NOT NULL,
  PRIMARY KEY("id_proveedor")
);

```

```

CREATE TABLE IF NOT EXISTS "Mercaderia" (
  "id_ingreso" INTEGER NOT NULL,
  "id_proveedor" INTEGER NOT NULL,
  "numero_comprobante" VARCHAR(50) NOT NULL,
  "fecha_ingreso" TIMESTAMP NOT NULL,
  "total_pagado" NUMERIC(10,2) NOT NULL,
  PRIMARY KEY("id_ingreso")
);

```

```

CREATE TABLE IF NOT EXISTS "Detalle_Ingreso" (

```

```

"id_detalle_ingreso" INTEGER NOT NULL,
"id_ingreso" INTEGER NOT NULL,
"id_producto" INTEGER NOT NULL,
"cantidad" INTEGER NOT NULL,
"costo_unitario" NUMERIC(10,2) NOT NULL,
"subtotal" NUMERIC(10,2) NOT NULL,
PRIMARY KEY("id_detalle_ingreso")
);
ALTER TABLE "Mascota"
ADD FOREIGN KEY("id_duenio") REFERENCES "Usuario"("id_usuario")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Cita"
ADD FOREIGN KEY("id_mascota") REFERENCES "Mascota"("id_mascota")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Cita"
ADD FOREIGN KEY("id_veterinario") REFERENCES "Usuario"("id_usuario")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Historial_Clinico"
ADD FOREIGN KEY("id_mascota") REFERENCES "Mascota"("id_mascota")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Historial_Clinico"
ADD FOREIGN KEY("id_cita") REFERENCES "Cita"("id_cita")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Dispositivo_IoT"
ADD FOREIGN KEY("id_mascota") REFERENCES "Mascota"("id_mascota")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Chat_Triage"
ADD FOREIGN KEY("id_mascota") REFERENCES "Mascota"("id_mascota")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Mensaje_Chat"
ADD FOREIGN KEY("id_chat") REFERENCES "Chat_Triage"("id_chat")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Factura"
ADD FOREIGN KEY("id_cliente") REFERENCES "Usuario"("id_usuario")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Detalle_Factura"
ADD FOREIGN KEY("id_factura") REFERENCES "Factura"("id_factura")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Detalle_Factura"
ADD FOREIGN KEY("id_producto") REFERENCES "Producto_Servicio"("id_producto")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Detalle_Factura"
ADD FOREIGN KEY("id_cita") REFERENCES "Cita"("id_cita")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Mercaderia"
ADD FOREIGN KEY("id_proveedor") REFERENCES "Proveedor"("id_proveedor")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Detalle_Ingreso"

```

```

ADD FOREIGN KEY("id_ingreso") REFERENCES "Mercaderia"("id_ingreso")
ON UPDATE NO ACTION ON DELETE NO ACTION;
ALTER TABLE "Detalle_Ingreso"
ADD FOREIGN KEY("id_producto") REFERENCES "Producto_Servicio"("id_producto")
ON UPDATE NO ACTION ON DELETE NO ACTION;

```

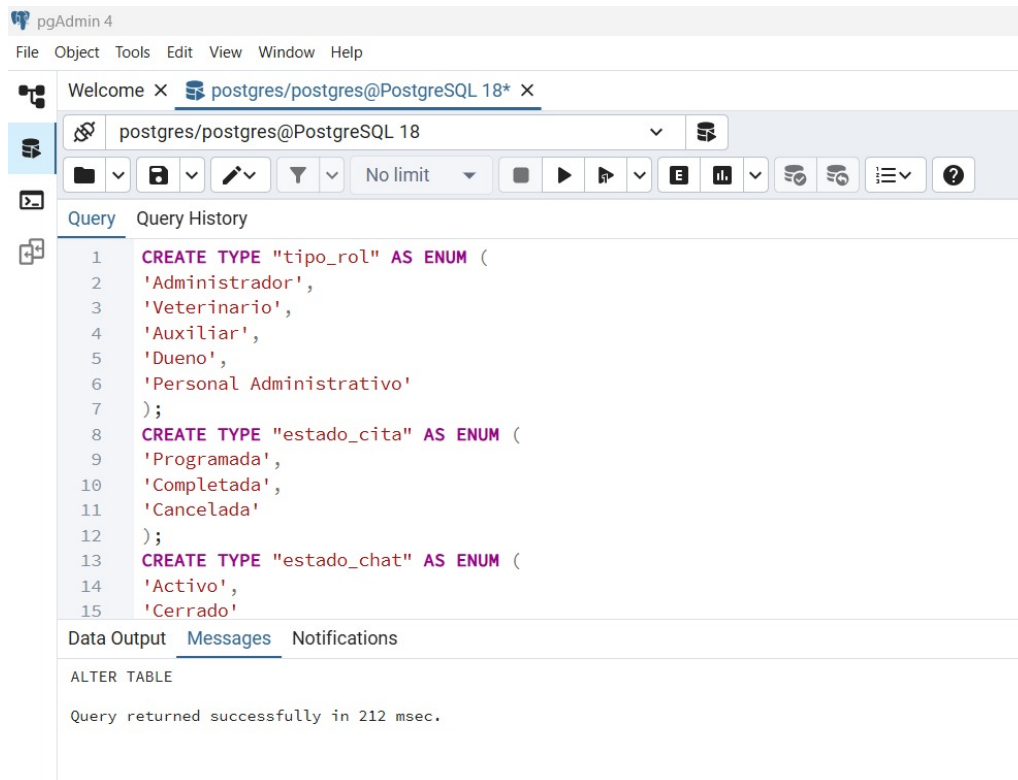


Figura 4: Base de datos creada correctamente

7. Diseño de interfaz: Wireframes

Para la conceptualización visual de BIOPET, se diseñaron wireframes detallando roles y flujos:

- **1. Login de Acceso**
 - **Rol:** Todos los usuarios (Visitante).
 - **Descripción:** Formulario de acceso unificado con campos de usuario, contraseña y botón de recuperación.
 - **Flujo:** Pantalla inicial. Al autenticar con éxito, redirige al Dashboard correspondiente.
- **2. Dashboard Veterinario/Administrativo**
 - **Rol:** Veterinario y Administrador.

- **Descripción:** Panel principal con tarjetas de métricas (mascotas registradas, citas del día, ingresos) y barra de navegación lateral.
 - **Flujo:** Accesible post-login. Permite navegar a módulos de Historial, Citas y Facturación.
- **3. Historial Clínico**
- **Rol:** Veterinario.
 - **Descripción:** Vista detallada con información del paciente, formulario de nueva consulta y panel lateral de recomendaciones de IA.
 - **Flujo:** Se llega desde el buscador de mascotas. Permite guardar datos y regresar al listado.
- **4. Gestión de Citas**
- **Rol:** Personal Administrativo y Dueño.
 - **Descripción:** Calendario interactivo y formulario lateral para agendar, confirmar o cancelar turnos médicos.
 - **Flujo:** Accesible desde el Dashboard. Abre modales de confirmación antes de impactar la BD.

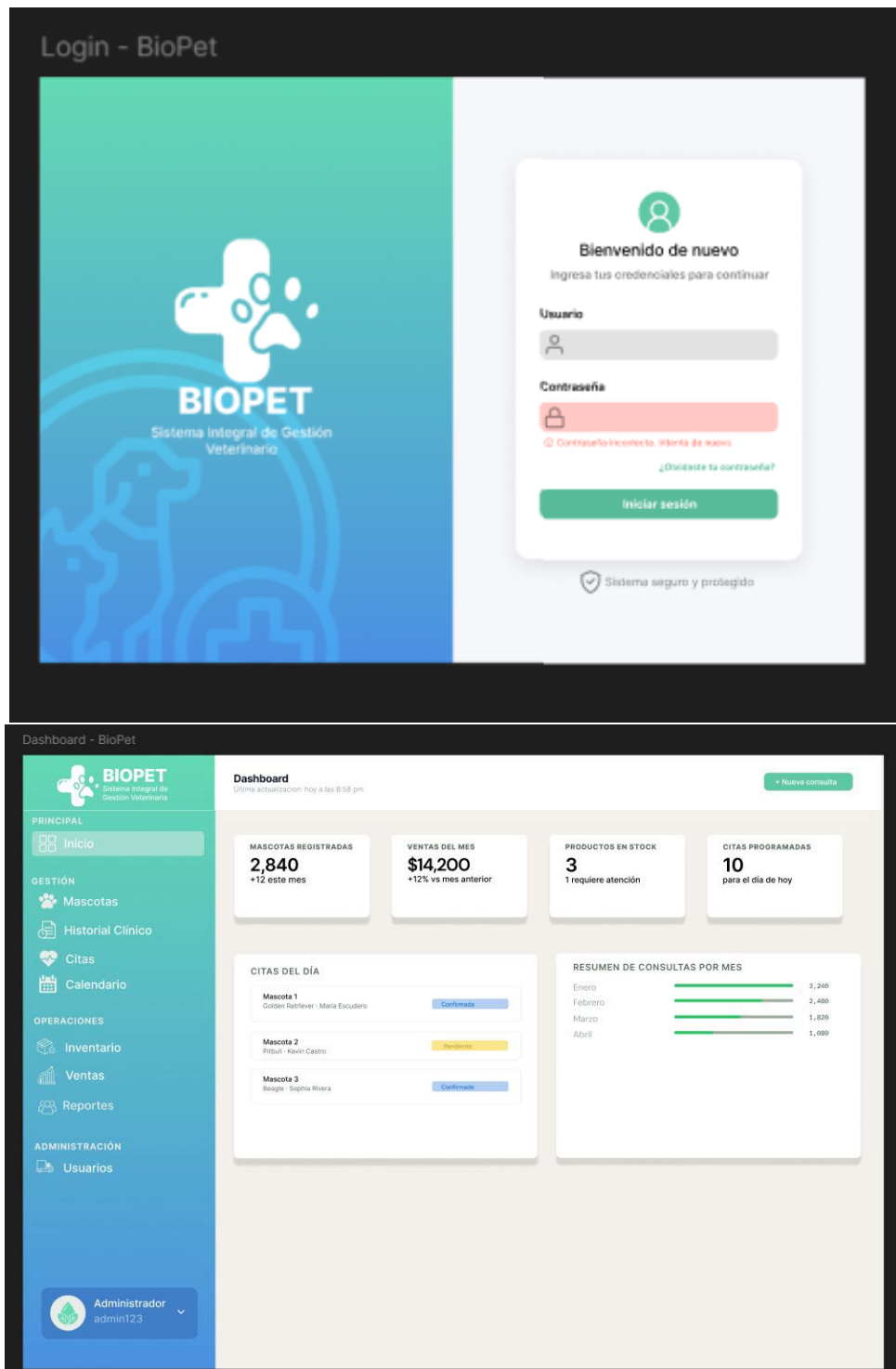


Figura 5: Wireframes de Login y Dashboard Veterinario

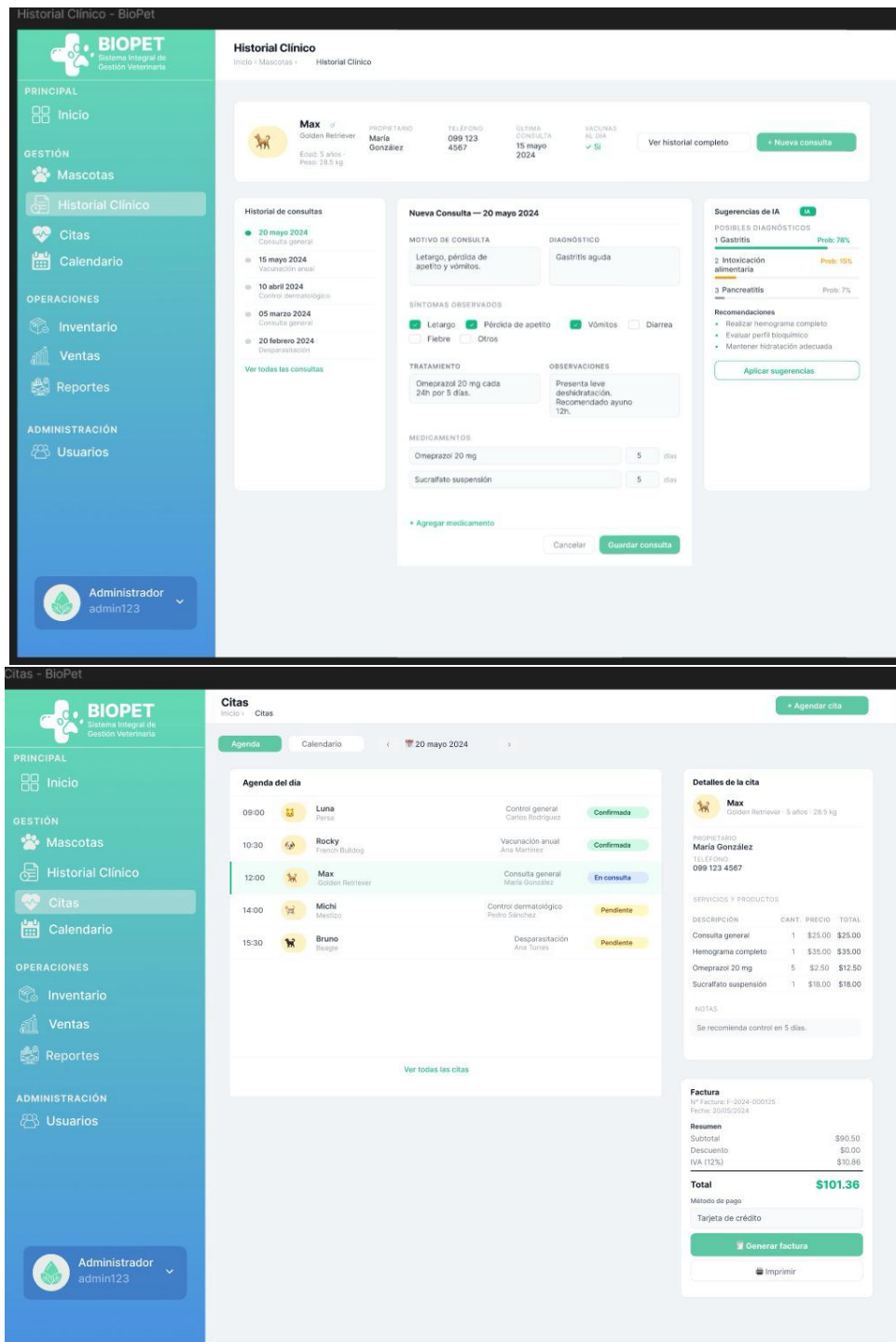


Figura 6: Wireframes de Historial Clínico y Citas

8. Plan de trabajo y roles

8.1. Cronograma de Hitos

Se utiliza un Diagrama de Gantt para planificar los hitos, coordinar el trabajo del equipo y asegurar la entrega en los tiempos establecidos [10].

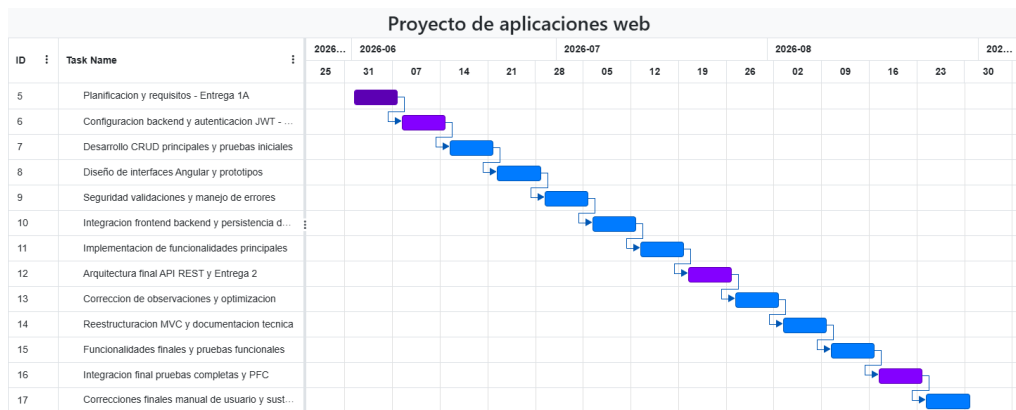


Figura 7: Diagrama de Gantt (Semanas 5 a 17)
(Aquí)

En la Tabla 10 se detallan algunos aspectos que no se observan en la figura 7, considerando que en al abrir el link del diagrama de gantt es posible visualizar más detalles.

Tabla 10: Cronograma de actividades del proyecto

Semana	Actividad principal	Responsable principal
5	Definición del proyecto, levantamiento de requisitos, alcance, casos de uso, planificación y diseño inicial. Elaboración y entrega del documento 1A.	Zaida Taipe
6	Configuración del entorno, creación del proyecto backend, diseño de base de datos, implementación de autenticación JWT y acceso a datos. Entrega 1B.	Jaime Mariscal
7	Desarrollo de módulos básicos del backend (CRUD principales) y pruebas de autenticación.	Jaime Mariscal
8	Diseño de interfaces en Angular, estructura de componentes, navegación y prototipos.	Fred Beltrán
9	Implementación de medidas de seguridad, validaciones, manejo de errores y pruebas de integración iniciales.	Jaime Mariscal
10	Implementación de persistencia de datos, conexión frontend-backend y gestión de sesiones.	Equipo completo
11	Desarrollo de funcionalidades principales del sistema y optimización de consultas a la base de datos.	Jaime Mariscal
12	Diseño de arquitectura definitiva, documentación de API REST, integración completa del frontend Angular y preparación de la Entrega 2.	Zaida Taipe
13	Corrección de observaciones de la Entrega 2, mejoras de rendimiento y escalabilidad.	Equipo completo
14	Reestructuración bajo patrón MVC, revisión de capas y documentación técnica del proyecto.	Zaida Taipe
15	Desarrollo de funcionalidades finales, pruebas funcionales y pruebas de aceptación.	Fred Beltrán
16	Implementación y documentación de servicios REST, integración final, pruebas completas y entrega del Sistema Completo (PFC).	Equipo completo

Semana	Actividad principal	Responsable principal
17	Corrección de errores detectados, preparación de presentación final, manual de usuario y sustentación.	Zaida Taipe

8.2. Asignación de roles

Integrante	Rol	Responsabilidades
Zaida Taipe	Líder Técnico	Revisión de arquitectura, integración IoT/IA, control de ramas en Git.
Fred Beltrán	Frontend	Maquetado responsivo, consumo de API, UI/UX.
Jaime Mariscal	Backend	APIs REST en ASP.NET Core, Base de datos y despliegue.

9. Estructura del repositorio

El repositorio en GitHub ha sido inicializado cumpliendo con la estructura requerida para la Entrega 1A. Todos los integrantes poseen acceso y han realizado commits iniciales.

Estructura principal del repositorio:

- **README.md:** Contiene la descripción general del proyecto BIOPET.
- **/.gitignore:** Configurado para ignorar binarios de Visual Studio y módulos pesados.
- **/.env.example:** Plantilla de variables de entorno para cadenas de conexión a PostgreSQL y secretos JWT.
- **/docs/:** Directorio para almacenar el ADR, el script DDL de la BD y los diagramas generados.
- **BIOPET.sln:** Solución principal del backend en ASP.NET Core 8.

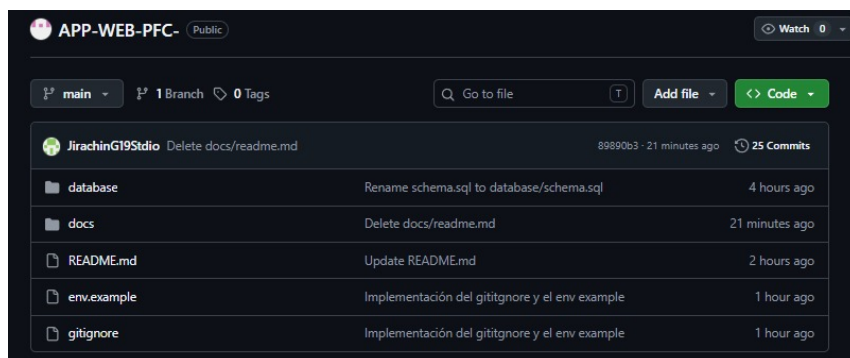


Figura 8: Estructura en el repositorio

Enlace: ([Repositorio Git de BIOPET](#))

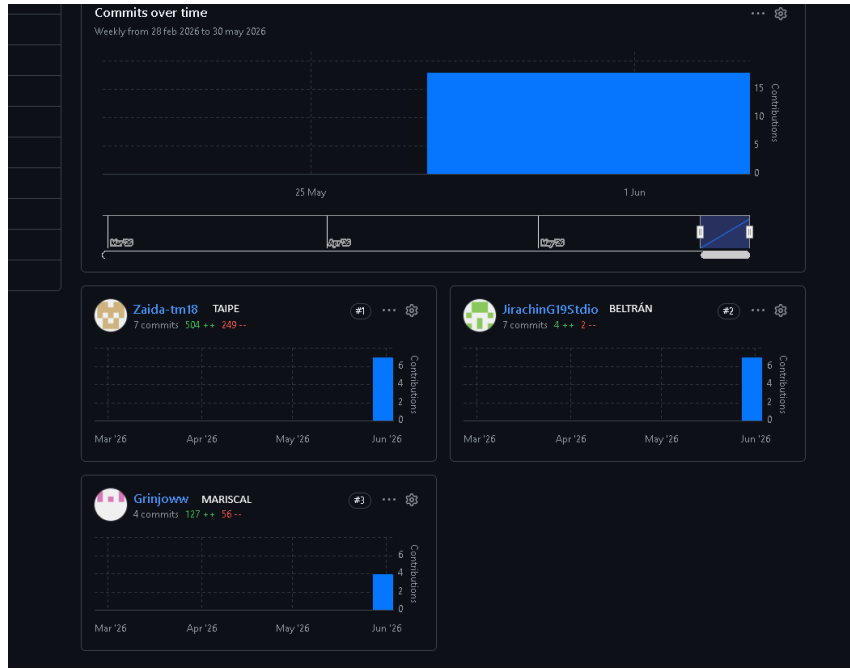


Figura 9: Primeros commits en el repositorio

Referencias

- [1] Szewczyk, M., & Skublewska-Paszkowska, M. (2025). Performance comparison of development frameworks in selected environments in REST API architecture. *Journal of Computer Sciences Institute*, 35, 121-128.
- [2] Islam, M. M., & Bhuiyan, Z. A. (2023). An Integrated Scalable Framework for Cloud and IoT Based Green Healthcare System. *IEEE Access*, 11, 22282-22295. DOI: 10.1109/ACCESS.2023.3250849
- [3] Mironov, V. V., & Mironov, K. V. (2025). MP-integrity in designing relational database models. *Ontology of Designing*, 15(4), 552-565. DOI: 10.18287/2223-9537-2025-15-4-552-565
- [4] Kushal, B. C., Barai, S., Timilsina, S., Lekhi, P., & Bhavani, B. G. (2026). EstateManager: A JWT-Secured Real Estate Platform with Role-Based Access Control. *International Journal for Research in Applied Science and Engineering Technology*, 14(2). DOI: 10.22214/ijraset.2026.77369
- [5] Alowais, S. A., Alghamdi, S. S., Alsuhebany, N., et al. (2023). Revolutionizing healthcare: the role of artificial intelligence in clinical practice. *BMC Medical Education*, 23(1), 689. DOI: 10.1186/s12909-023-04698-z
- [6] Buchgeher, G., Schöberl, S., Geist, V., Dorninger, B., Haindl, P., & Weinreich, R. (2023). Using Architecture Decision Records in Open Source Projects—An MSR Study on GitHub. *IEEE Access*, 11, 63740-63755. DOI: 10.1109/ACCESS.2023.3287654
- [7] Szczepanik, K., & Chudziak, J. A. (2025). Collaborative LLM Agents for C4 Software Architecture Design Automation. *arXiv preprint arXiv:2510.22787*.

- [8] Shreda, Q. A., & Hanani, A. A. (2021). Identifying Non-Functional Requirements From Unconstrained Documents Using Natural Language Processing and Machine Learning Approaches. *IEEE Access*, 9, 3052921. DOI: 10.1109/ACCESS.2021.3052921
- [9] Singh, A. (n.d.). Evolutionary Architectures in Web Applications: A Comprehensive Study of Client-Server, Multi-Tier, and Service-Oriented Approaches. *International Journal for Multidisciplinary Research (IJFMR)*.
- [10] Novak, Y., Sobko, Y., & Sumariuk, O. (2023). Expediency of using Gantt charts in modern construction. *Modern Problems of Architecture and Urban Planning*, 65, 291-300. DOI: 10.32347/2077-3455.2023.65.291-300